ECE60827 Hardware Simulation 1
4/16/2023
Jerry Liu

Data Point 1:

Total Simulator Instructions:

	b+tree	backprop	bfs	gauss	hotspot	lud	myocyte	nn	nw	srad
SASS	407021552	148832592	379371108	144833960	360616472	25984224	1669062	1327636	146325504	967185678
PTX	380891849	161546592	541515763	225527328	360179256	32201872	798310	1201052	99791872	1000002640

Total Simulation Cycles:

	b+tree	backprop	bfs	gauss	hotspot	lud	myocyte	nn	nw	srad
SASS	126760	62228	927878	2492020	62061	665434	6132410	6389	2523030	751525
PTX	125288	59116	971311	2577127	67178	841654	6412507	6077	4294497	1245332

Observations:

Overall, it appears that SASS, on average, runs fewer cycles and instructions than the PTX simulations. Though there are some exceptions where the numbers are similar, they imply that the SASS ISA requires fewer instructions than the PTX ISA to perform the same computations. As a result, they also imply that the SASS ISA is more optimized since it can complete the same benchmarks with fewer cycles and instructions. This conclusion is more or less expected since the PTX ISA is a more general, "higher level" virtual instruction set, while SASS is what will ultimately be compiled into binary code (and is therefore more optimized to the specific architecture/use case).

Data Point 2:

For average SIMD efficiency, two metrics were calculated. The first metric is average active SIMD efficiency where at least 1 thread of a warp was issued every cycle (control flow divergence), and the second metric is average overall SIMD efficiency (including cycles where no instructions were issued). Note that stall cycles were not included for either metric because stalls shouldn't technically introduce bubbles into the SIMD datapath (since the pipeline is stalled). The generalized equation for the calculation is shown below:

$$SIMD\ \%\ Effiency = \frac{Total\ active\ threads\ (\sum W(N) * N)}{Total\ warps/cycles\ (\sum W(N)) * Warp\ size} * 100$$

Average SIMD Efficiency:

	b+tree	backprop	bfs	gauss	hotspot	lud	myocyte	nn	nw	srad
Active	93.93%	91.65%	35.76%	96.29%	91.20%	83.30%	3.26%	99.96%	35.51%	88.97%
Overall	50.18%	41.53%	5.16%	13.03%	86.28%	7.58%	0.16%	11.77%	1.71%	56.35%

ECE60827 Hardware Simulation 1
4/16/2023
Jerry Liu

As for memory divergence, the main metric is the source (L1D) metric. This metric using the following data for the number of memory accesses:

m_accessq.size() - starting_queue_size

It should be noted that the same metric was initially gathered using the number of L1D accesses (L1D_total_cache_accesses), but this metric may be incorrect if there are other accesses (i.e. local accesses) to the L1D. However, in practice these two statistics were identical for all applications run, so the collected data remains valid. An additional DRAM metric was also gathered for comparison with the L1 cache. This statistic measures the total number of accesses to the DRAM subsystem (gpgpu_n_mem_write/read_global), and it acts as a nice point of comparison to see how well the caching system works for different workloads. Both metrics use the same (custom) statistic to determine the number of memory instructions (gpgpu_n_global_ops). Like the L1D_total_cache_accesses, this metric was determined to be equivalent to counting the total number of times the L1D was issued a global memory operation (# of times m_accessq.size() - starting_queue_size is executed).

Average Memory Accesses Per Memory Instruction:

	b+tree	backprop	bfs	gauss	hotspot	lud	myocyte	nn	nw	srad
Source	2.205	3.519	2.838	12.515	5.397	3.598	1.000	6.664	3.200	3.962
DRAM	0.686	2.298	1.746	6.201	5.325	3.585	0.555	3.998	3.171	2.786

Total Program IPC:

	b+tree	backprop	bfs	gauss	hotspot	lud	myocyte	nn	nw	srad
IPC	3211.75	2393.58	401.50	58.13	5852.45	39.07	0.27	207.80	57.99	1284.61

Observations:

For purposes of comparison and analysis, the SIMD active efficiency metric and the source divergence metric will be used as they best fit the desired testing metrics laid out in the project guidelines. With that in mind, these metrics appear to indicate that, generally, the more divergence (lower SIMD efficiency and or more memory accesses/inst) there is in a given program, the lower the IPC. For a majority of the applications, this observation holds true, though there are some small exceptions such as hotspot. Even though hotspot has fairly mediocre divergence metrics, it has the highest IPC of all the benchmarks. This could be caused by the other simulation metrics that were not included in the divergence statistics, such as total occupancy or stall cycles. In the case of hotspot, it has a fairly high total occupancy of about 84%, and a relatively low ratio of stalled cycles to total program cycles.